

Ensamblador

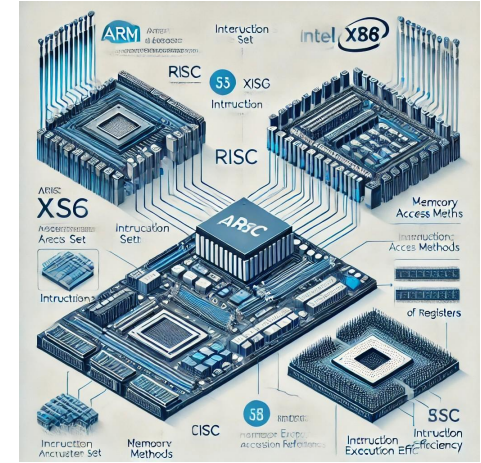
arm

Modos de Direcccionamiento

Ejemplos ARM -Advanced RISC Machine-

Direccionamiento

- ¿Cómo hacemos el acceso a memoria?
- Distintas forma de acceder a los datos en memoria
- No se puede operar directamente con los datos en memoria



¿Qué es el direccionamiento?

Es la forma en que un procesador accede a los datos en memoria

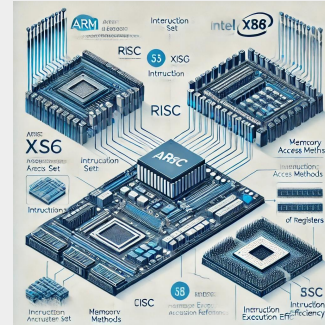
Las distintas **formas** de acceder a los parámetros de una instrucción se llaman **modos de direccionamiento**

Forma en que una instrucción determina los operandos con los que trabajará

Importancia en arquitecturas RISC como ARM

Optimiza rendimiento y uso de memoria

En ARM, debido a su arquitectura **Load/Store**, solo se puede operar directamente con datos en registros



Tipos de direccionamiento

- 1 **Inmediato:** Operando es un valor constante
- 2 **A Registro:** Operando es un registro
- 3 **Relativo a registro:** Registro almacena dirección de memoria
- 4 **Relativo a registro Con Desplazamiento:** Registro base + desplazamiento
- 5 **Direccionamiento relativo a base más índice**
- 6 **En las instrucciones de salto incondicional y condicional:** direccionamiento absoluto. Uso de **b** *etiqueta*, bxx

Inmediato

Uno de los **operandos** es un **valor constante**, simboliza #12 y en el caso de ser un hexadecimal #0x8

```
mov r1, #12
```

```
add r0, #0x9
```

A Registro

Los **operandos** son **registros**, la operación se resuelve con el contenido de los registros

```
mov r1, r2
```

```
add r1, r3
```

Direccionamiento relativo a registro

Para aplicar este direccionamiento, primero debemos obtener una dirección de memoria

```
.data
dato: .word 1500      // Tenemos definido dato con un valor inicial de 1500
                        // dato ocupa una posición en memoria
.text
.global main:
main:
    ldr r1, =dato      // al acceder a dato con el =dato, lo que logramos es recuperar en r1 la dirección de
                        // memoria donde se encuentra dato

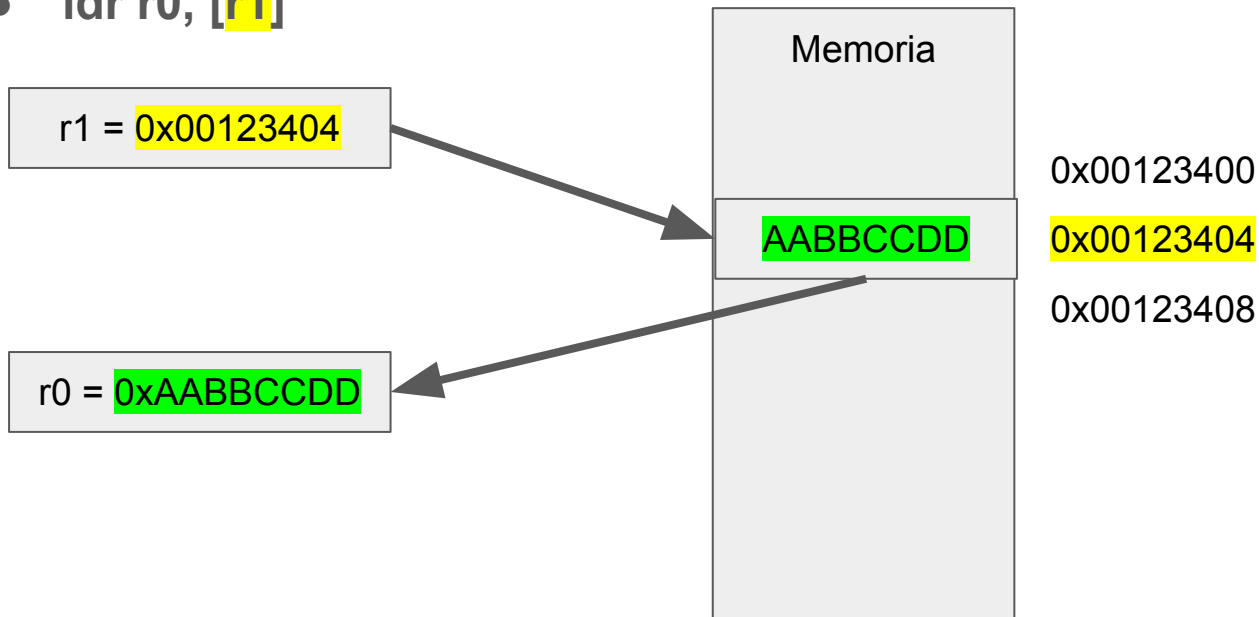
    ldr r2,[r1]        //Obtenemos en r2 el contenido de dato, o lo que es lo mismo de la dirección de memoria contenida en r1
```

La instrucción ldr es la que me permite acceder a la posición de memoria contenida en r1 y el contenido de esa posición es cargado en el registro r2

ldr Rd, [Rx, #offset]

Direccionamiento relativo a registro

- `ldr r0, [r1]`



Direccionamiento relativo a registro

- En la arquitectura ARM solo las instrucciones de LOAD/STORE pueden acceder a memoria
- La forma más simple es acceder a la posición de memoria apuntada por el contenido de un registro, por ejemplo: **ldr r0, [r1]**
- Previamente debe almacenarse en r1 la dirección de memoria a la que se quiere acceder
- El valor que se copia en r0 al ejecutar la instrucción ldr es el contenido de la memoria en la posición indicada por un registro (en este caso r1)
- También podemos llamar a este modo de direccionamiento "Indirecto"
- Otros procesadores permiten acceder a una posición de memoria dada por una constante. Eso sería un modo de direccionamiento "Directo"

Direccionamiento relativo a registro con desplazamiento

ldr Rd, [Rx, #offset]

En #offset tenemos un desplazamiento respecto del contenido del registro Rn

carga el contenido de la memoria al registro r2

ldr r2,[r1,#5]

Puede ser positivo o negativo

ldr r2,[r1,#-2]

La inversa de esta instrucción es **str r2,[r1,#5]**

Las instrucciones ldr y str sirven para trabajar con datos de tamaño word

Si tenemos que trabajar con instrucciones byte usamos **ldrb y strb**

Para media word (2 bytes) **ldrh y strh**

Instrucción STR

STR (Store Register) almacena un word de un registro a una dirección de memoria.

STR{<cond>} <Rd>, <addressing_mode>

donde:

<cond> es la condición para ejecutar la instrucción, es opcional

<Rd> especifica el registro fuente

<addressing_mode> es el modo de direccionamiento (forma de acceder a los datos, ver clase de modos de direccionamiento).

Instrucción STR

Ejemplo

```
.data
```

```
var1: .word 0
```

```
.text
```

```
.global main
```

```
main :
```

```
    mov r1, #128 // recordar que puedo mover immediatos de 0 a 255
```

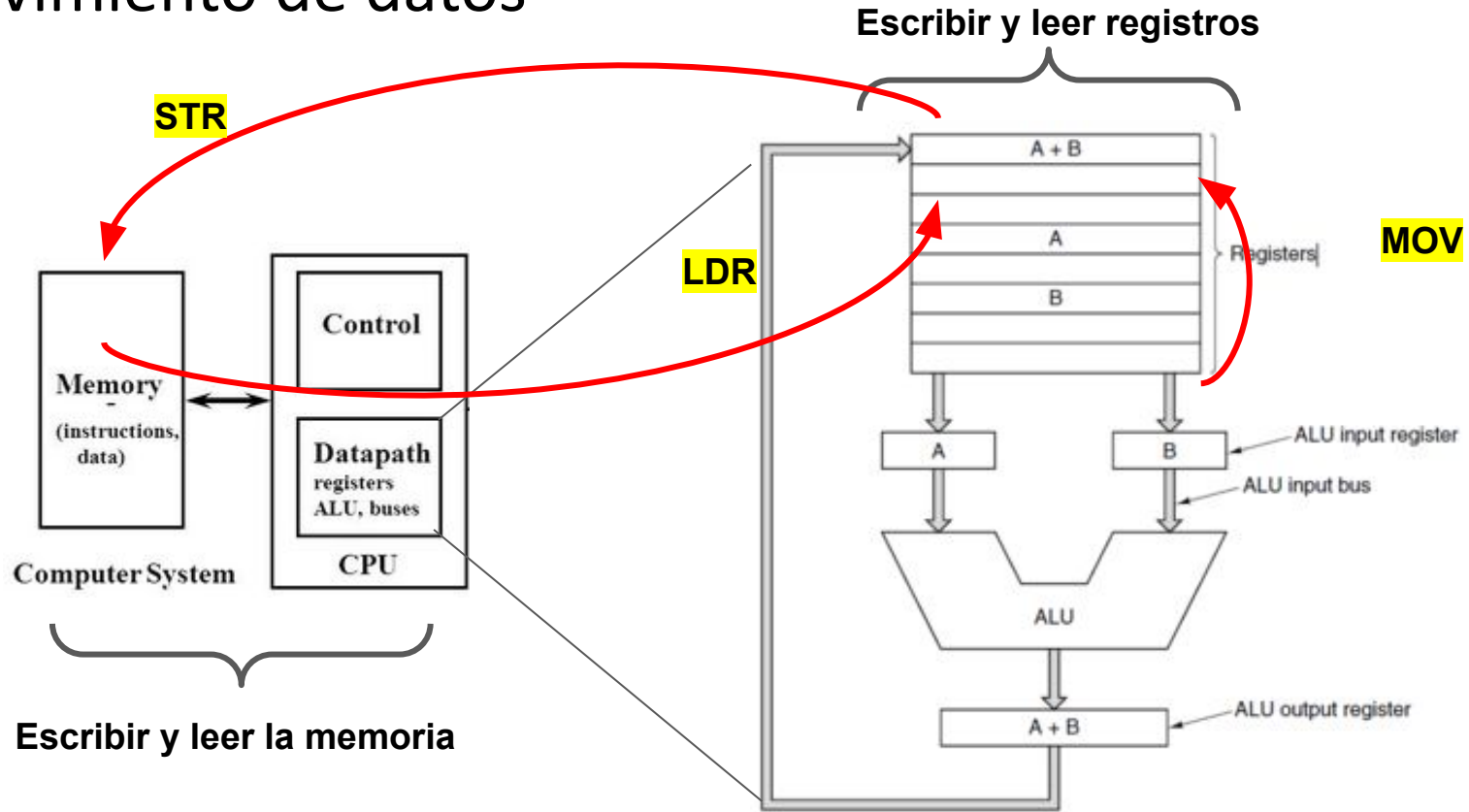
```
    ldr r0, =var1 // r0 <- & var1 , r0 tiene la dirección de memoria de var1
```

```
    str r1, [r0] // var1 <- 128, guardo el contenido de r1 en la dirección indicada por r0
```

Direccionamiento relativo a registro con desplazamiento

- La dirección efectiva del operando se obtiene sumando el contenido de un registro y un desplazamiento constante.
- Por ejemplo en la instrucción **str r2, [r3, #8]** a la dirección de memoria contenida en r3 se le suma 8.
- Este modo de direccionamiento es útil para acceder a datos ubicados en una posición relativa a una dirección de memoria conocida. Por ejemplo el elemento siguiente o anterior en un vector, o un componente de una estructura (tipo struct de C).
- El valor del registro base (r3) no se modifica.
- El desplazamiento admite valores de hasta 12 bits.

Movimiento de datos



Direccionamiento relativo a base más índice

- En este modo de direccionamiento, la ubicación del parámetro se obtiene sumando el valor de dos registros.
- Por ejemplo la instrucción **ldr r2, [r3, r4]** carga en r2 el contenido de la memoria en la posición indicada por $r3 + r4$.
- Este modo de direccionamiento es útil para recorrer acceder a estructuras complejas. Por ejemplo si un registro apunta al primer elemento de un vector, usando un segundo registro se puede recorrer dicho vector.

Direccionamiento relativo a base más índice desplazado

- La arquitectura del set de instrucciones ARM admite agregar un desplazamiento o rotación que se aplica al segundo registro en las instrucciones ldr y str.
- Por ejemplo la instrucción **ldr r2, [r3, r4, LSL #2]** aplica un desplazamiento lógico a izquierda de 2 bits al registro r4 antes de sumarlo con r3.
- Notar que el desplazamiento a la izquierda de dos bits equivale a multiplicar por 4 el registro r4.
- Esto es útil para recorrer un vector de words (cada elemento ocupa 4 bytes), ya que puede usarse un índice natural almacenado en r4.
- El valor que se almacena en r2 al ejecutar esta instrucción es el contenido de la memoria en la posición dada por $r3 + r4 * 4$.

En síntesis

- Vimos que en la arquitectura ARM hay distintos tipos de instrucciones
 - Movimiento o copia de datos
 - Carga o almacenamiento de datos en memoria
 - Instrucciones aritméticas o lógicas
- Estas instrucciones pueden tener uno, dos, o tres parámetros
 - `mov r1, #18`
 - `ldr r0, [r2]`
 - `add r3, r4, #1`
- Las instrucciones traducidas a código máquina tienen un tamaño fijo: 32 bits
En esos 32 bits se codifican, entre otras cosas, los parámetros de la instrucción
- Los **modos de direccionamiento** son las distintas maneras de referirse a la ubicación de cada parámetro de una instrucción

En Código de Máquina - Direccionamiento inmediato

- El valor del parámetro es una constante, que forma parte de la instrucción.
- Por ejemplo la instrucción **add r0, r1, #1** traducida a código máquina corresponde a los siguientes 4 bytes:

E2 81 00 01 = 11100010100000010000000000000001

- Los 12 bits menos significativos (resaltado amarillo) almacenan el valor del inmediato. 8 bits para el valor y 4 bits para codificar un desplazamiento.
- Esto implica que el rango de valores inmediatos que podemos usar está limitado a 8 bits.

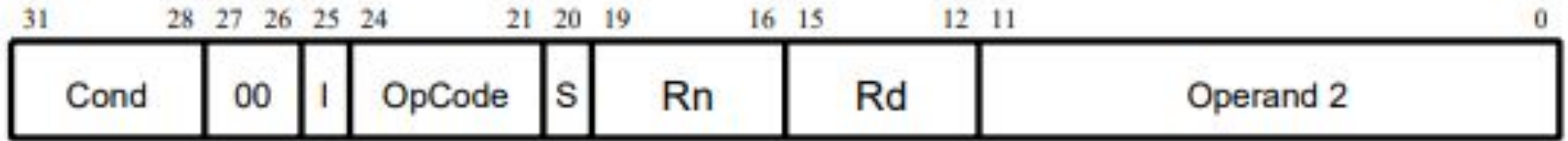
En Código de Máquina - Direccionamiento a registro

- En este caso el valor del parámetro está almacenado en un registro. Basta con identificar cuál es el registro.
- Siguiendo con la instrucción **add r0, r1, #1** podemos ver que el parámetro r1 está codificado de la siguiente manera

E2 81 00 01 = 11100010100000010000000000000001

- Los cuatro bits resaltados en **amarillo** representan el segundo parámetro, r1 en este caso.
- El registro r0 está codificado en los 4 bits resaltados en **naranja**

Decodificando **add r0, r1, #1**



E2810001 = 11100010100000010000000000000001

- El inmediato **#1** está contenido en los bits Operand 2
- **Rd** codifica el primer parámetro (registro destino)
- **Rn** codifica el segundo parámetro
- El patrón de bits presente en **OpCode** (verde) identifica la instrucción add.

Es posible generar un listado con el código máquina de cada instrucción ensamblando con la opción `as -a`

Referencias

- Libro de Prácticas
<https://riuma.uma.es/xmlui/bitstream/handle/10630/10214/LibroDePracticas.pdf> sección 2.1.1
- ARMv6-M Architecture Reference Manual
https://github.com/pyTony/rp2040/blob/main/DDI0419E_armv6m_arm.pdf
- Formato instrucción add - ARM7DI Datasheet
- <https://developer.arm.com/documentation/ddi0419/c/>
http://infocenter.arm.com/help/topic/com.arm.doc.ddi0027d/DDI0027D_7di_ds.pdf p. 29
- <https://developer.arm.com/documentation/dui0068/b/ARM-Instruction-Reference?lang=en>